Tom, Pierre
& Imane

Prometheus

Grafana

debian 13

Objectif – superviser les ordinateurs

Mission – Installation du service de supervision Prometheus pour faire de la supervision active et passive au sein d'un réseau.

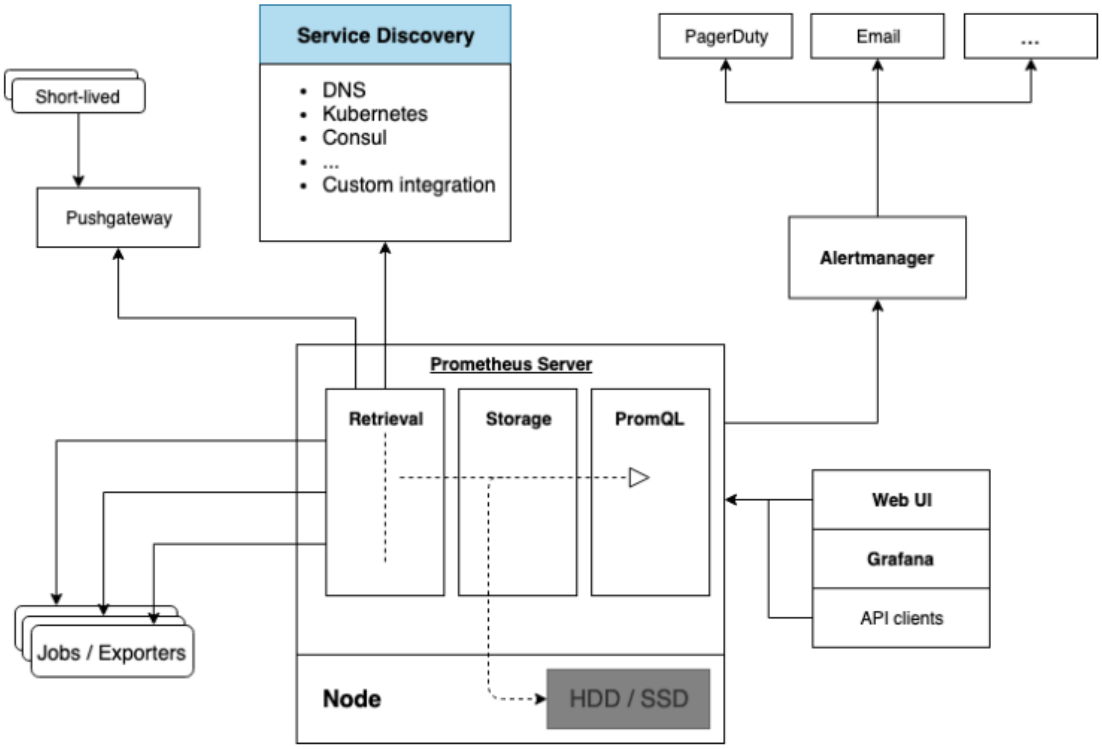
Sommaire :

I – Introduction 2
II – Installation 3
1. Mise à jour du système 3
2. Création de l'utilisateur Prometheus 3
3. Téléchargement de Prometheus 4
4. Installation des fichiers 5
III – Configuration de Prometheus 6
IV - Création du service systemd 6
1. Fichier et contenu 6
2. Démarrage et activation 7
3. Debug 8
V –Client Windows 11 9
1. Installation 10
2. Autoriser le pare-feu Windows (PowerShell) 11
3. Déclarer sur Debian 12

I – Introduction

Lien : https://blog.ippon.fr/2019/03/29/superviser-une-infrastructure-avec-prometheus-part-1-fonctionnement/

Prometheus est un outil de supervision moderne utilisé pour collecter et analyser des métriques en temps réel. Il est particulièrement adapté aux environnements cloud, aux conteneurs et à **Kubernetes** (= gère les applications) grâce à son modèle de **collecte « pull »** (= va chercher les données), sa grande performance et son intégration avec de nombreux exporters. Couplé à **Grafana**(= visualisation de données) , il permet une visualisation claire et efficace de l'état des systèmes et des applications.



II – Installation

Lien : https://balajisomasale.hashnode.dev/how-to-install-prometheus-on-windows-linux-and-macos

1. Mise à jour du système

Sur une machine Debian 13, on se log en mode root, tout en ne pas oubliant pas les mises à jour.

Commande : su -
Commande : apt update && apt upgrade –y

2. Création de l'utilisateur Prometheus

On fait la création de l'utilisateur de Prometheus, par sécurité on fait un utilisateur dédié.

Commande : useradd - - no-create-home - - shell /usr/sbin/nologin prometheus

```
root@debian13:~# useradd --no-create-home --shell /usr/sbin/nologin prometheus
```

3. Téléchargement de Prometheus

Puis on va faire un téléchargement de Prometheus depuis GitHub.

Commande : apt install git -y

```
root@debian13:~# apt install git
```

Puis on va dans **cd /tmp** pour installer prometheus. On va sur leur GitHub/site afin de prendre la dernière.

Lien Prometheus : https://prometheus.io/download/
Lien du GitHub : https://github.com/prometheus/prometheus/releases
Commande : wget https://github.com/prometheus/prometheus/releases/latest/download/prometheus-3.8.1.linux-amd64.tar.gz

```
root@debian13:/tmp# wget https://github.com/prometheus/prometheus/releases/latest/download/prometheus-3.8.1.linux-amd64.tar.gz
```

Ensuite, on va extraire le fichier qu'on a téléchargé.

Commande : tar xvf prometheus-3.8.1- linux-386.targ.gz

```
root@debian13:/tmp# tar xvf prometheus-3.8.1.linux-amd64.tar.gz
```

On vérifie que le fichier est bien installé avec : **ls**.

```
root@debian13:/tmp/prometheus-3.8.1.linux-386# ls
LICENSE  NOTICE  prometheus  prometheus.yml  promtool
```

4. Installation des fichiers

On va copier/déplacer le fichier **prometheus** et **promtool** sur le /usr/local/bin/ puis chown

```
root@debian13:/tmp# mv prometheus-3.8.1.linux-amd64/prometheus /usr/local/bin
root@debian13:/tmp# mv prometheus-3.8.1.linux-amd64/promtool /usr/local/bin/
```

```
root@debian13:/tmp/prometheus-3.8.1.linux-amd64# chown prometheus:prometheus /usr/local/bin/prometheus /usr/local/bin/promtool
```

Ensuite, on va **configurer le dossier** dans la création de son dossier **mkdir /etc/prometheus/** en copiant la commande.

/!\ création du dossier /etc/prometheus/ s'il n'est pas déjà créer /!

```
root@debian13:/tmp/prometheus-3.8.1.linux-amd64# cp prometheus.yml /etc/prometheus/
root@debian13:/tmp/prometheus-3.8.1.linux-amd64# chown -R prometheus:prometheus /etc/prometheus/
```

Commande : cp prometheus.yml /etc/prometheus
Commande : chown –R prometheuse:prometheus /etc/promjournalctl -etheus

Puis, le **dossier de données** tout comme le dossier de configuration. On crée le dossier dans **/var/lib/prometheus** , on donne les droits des dossiers et du fichiers.

/!\ création du dossier /var/lib/prometheus s'il n'est pas déjà créer /!

Commande :mkdir /var/lib/prometheus
Commande : chown prometheus:prometheus /var/lib/prometheus

```
root@debian13:/tmp/prometheus-3.8.1.linux-amd64# chown prometheus:prometheus /var/lib/prometheus/
```

III – Configuration de Prometheus

Dans le fichier principal, on va le configurer. On va changer le temps d'évaluation pour que ça soit tous les 1min et non toutes les 15s

Commande : nano /etc/prometheus/prometheus.yml

```yaml
global:
  scrape_interval: 60s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
  evaluation_interval: 60s # Evaluate rules every 15 seconds. The default is every 1 minute.
  # scrape_timeout is set to the global default (10s).

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

  static_configs:
    - targets: ["localhost:9090"]
     # The label name is added as a label `label_name=<label_value>` to any timeseries scraped from this config.
      labels:
        app: "prometheus"
```

Enfin, on vérifie que la configuration est bien.

Commande : promtool check config /etc/prometheus/prometheus.yml

```
root@debian13:/tmp/prometheus-3.8.1.linux-amd64# promtool check config /etc/prometheus/prometheus.yml
Checking /etc/prometheus/prometheus.yml
 SUCCESS: /etc/prometheus/prometheus.yml is valid prometheus config file syntax
```

IV - Création du service systemd

1. Fichier et contenu

On va créer un fichier vide systemd pour activer et démarer le service prometheus.

Commande : nano /etc/systemd/system/prometheus.service

```
[Unit]
Description=Prometheus Monitoring
Wants=network-online.target
After=network-online.target

[Service]
StartLimitInterval=5min
StartLimitBurst=10
RestartSec=30s

User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
        --config.file /etc/prometheus/prometheus.yml \
        --storage.tsdb.path /var/lib/prometheus \
        --web.console.templates=/etc/prometheus/consoles \
        --web.console.libraries=/etc/prometheus/console_libraries \
        --web.listen-address=0.0.0.0:9090

Restart=always

[Install]
WantedBy=multi-user.target
```

2. Démarrage et activation

On va faire le démarrage du fichier qu'on a créé.

Commande : systemctl daemon-reload

```
root@debian13:/tmp/prometheus-3.8.1.linux-386# systemctl daemon-reload
```

Puis on va autoriser le système, il va créer un chemin pour l'acceptation du démarrage qu'on a mis sur le fichier **(nano /etc/systemd/system/prometheus.service)**

Commande : systemctl enable prometheus

```
root@debian13:/tmp/prometheus-3.8.1.linux-386# systemctl enable prometheus
Created symlink '/etc/systemd/system/multi-user.target.wants/prometheus.service' → '/etc/systemd/system/prometheus.service'.
```

Enfin, on va activer prometheus.

Commande : systemctl start prometheus

```
root@debian13:/tmp/prometheus-3.8.1.linux-386# systemctl start prometheus
```

On regarde le status de notre service pour voir s'il est bien actif.

Commande : systemctl status prometheus.servie

```
● prometheus.service - Prometheus Monitoring
     Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; preset: enabled)
     Active: activating (auto-restart) (Result: exit-code) since Thu 2025-12-18 14:22:04 CET; 6s ago
 Invocation: d113b765939a46eba8fae7e96f269b53
    Process: 3708 ExecStart=/usr/local/bin/prometheus \ (code=exited, status=1/FAILURE)
   Main PID: 3708 (code=exited, status=1/FAILURE)
   Mem peak: 11.3M
        CPU: 39ms

déc. 18 14:22:04 debian13 systemd[1]: prometheus.service: Main process exited, code=exited, status=1/FAILURE
déc. 18 14:22:04 debian13 systemd[1]: prometheus.service: Failed with result 'exit-code'.
```

3. Debug

On va vérifier si c'est mon chemin de prometheus.

Commande : which prometheus

```
root@debian13:~# which prometheus
/usr/local/bin/prometheus
```

Je regarde s'il a les droits sur les fichiers.

Commande : chown –R prometheus:prometheus /var/lib/prometheus

Enfin au niveau du fichier, on va virer les \ en cas de problème de compréhension et on enlève certaines lignes qui sont en trop.

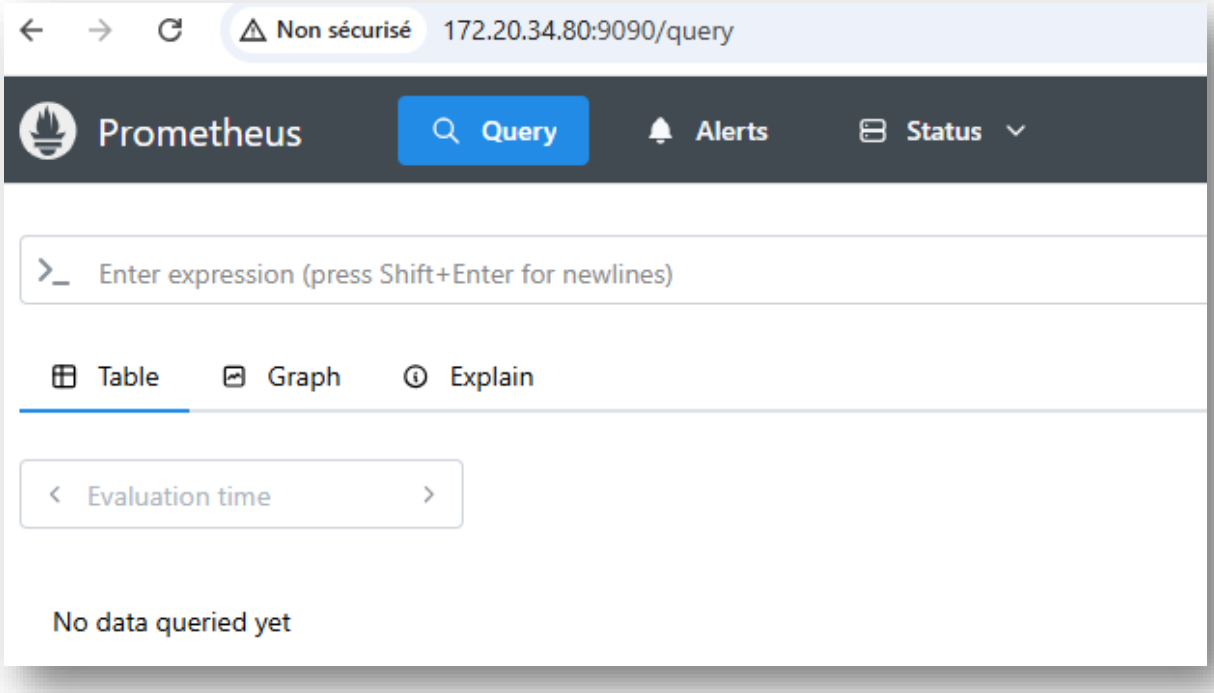
```
[Unit]
Description=Prometheus Monitoring
Wants=network-online.target
After=network-online.target

[Service]
StartLimitInterval=5min
StartLimitBurst=10
RestartSec=30s

User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.path=/var/lib/prometheus

Restart=always

[Install]
WantedBy=multi-user.target
```

On redémarre le système.

Commande : systemctl daemon-reload
Commande : systemctl restart prometheus
Commande : systemctl status prometheus

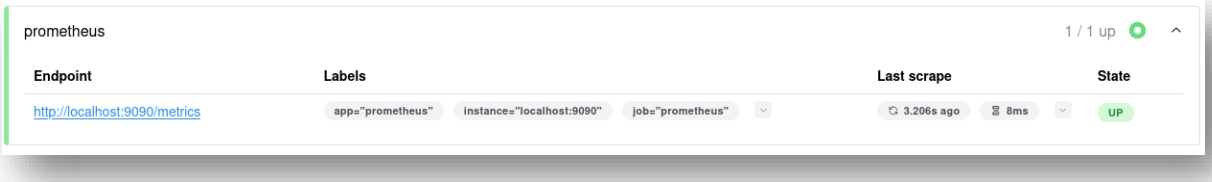
```
● prometheus.service - Prometheus Monitoring
     Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-12-18 14:42:48 CET; 11s ago
 Invocation: 34747509d260410f993178be9b654d35
   Main PID: 4767 (prometheus)
      Tasks: 8 (limit: 9451)
     Memory: 17.5M (peak: 17.9M)
        CPU: 90ms
     CGroup: /system.slice/prometheus.service
             └─4767 /usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.path=/var/lib/prometheus
```

On peut enfin se connecter sur notre serveur :
http://localhost:9090
http://172.20.34.80:9090



V –Client Windows 11

On vérifie que Prometheus est OK. On va voir dans son statut > Target pour voir s'il est UP.

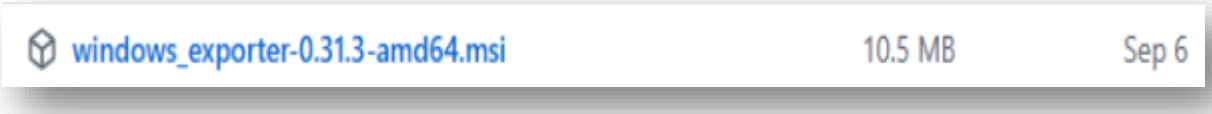


1. Installation

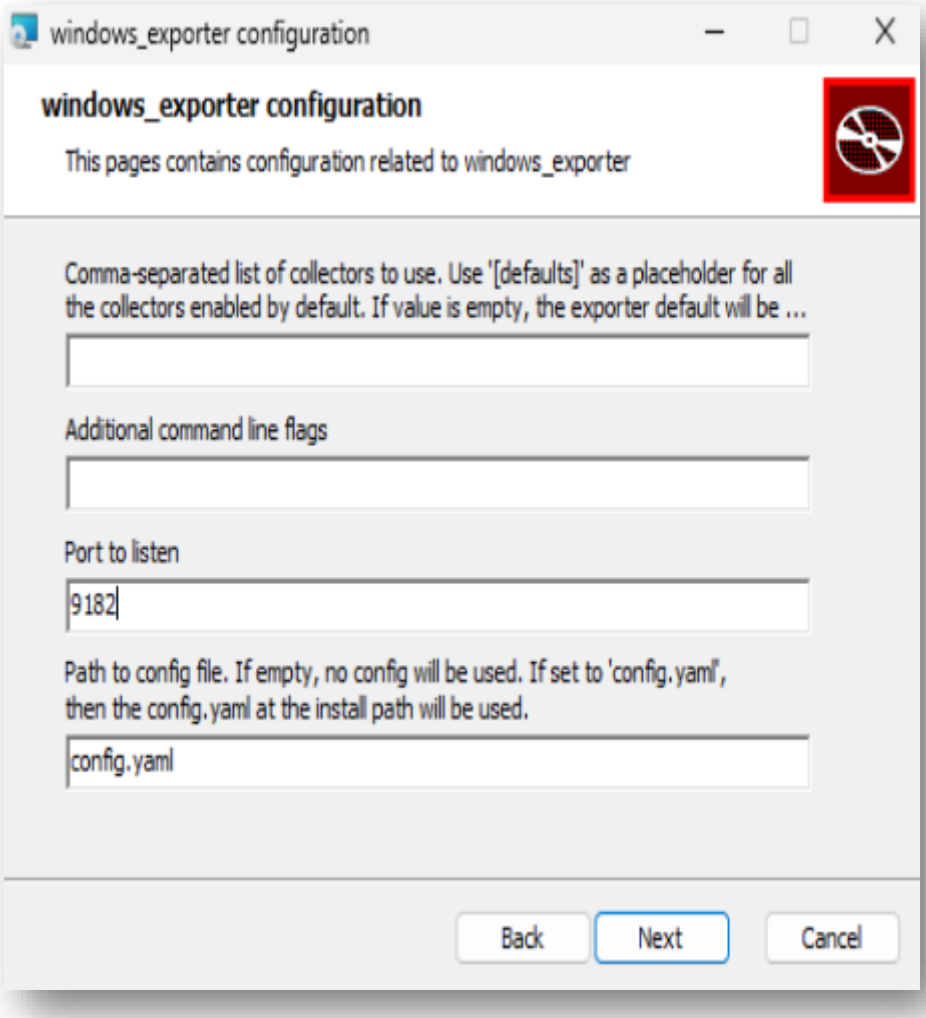
Ensuite, sur notre machine win11 on va installer l'exporter Windows 11.

Lien : https://github.com/prometheus-community/windows_exporter/releases

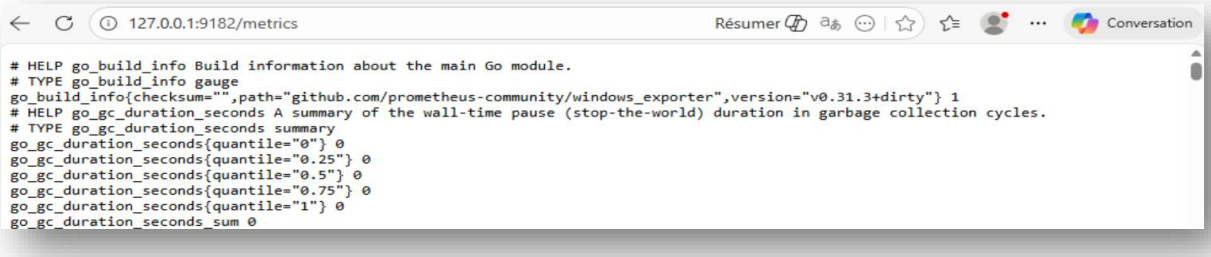
On télécharge le fichier **.msi**



Pour l'installation, on prend 9182 comme port d'écoute.



On va faire un **test local** sur le navigateur de notre machine.
http://127.0.0.1:9182/metrics



2. Autoriser le pare-feu Windows (PowerShell)

On va autoriser le port 9182 dans **PowerShell.ISE** en mode administrateur. On autorise les scripts.

```
PS C:\Windows\system32> Set-ExecutionPolicy Unrestricted
```

On **ouvre le port 9182** avec un nommage pour notre règle de pare-feu:

```
New-NetFirewallRule -DisplayName "Prometheus windows_exporter" `
    -Direction Inbound -Protocol TCP -LocalPort 9182 -Action Allow
```

```
PS C:\Windows\system32> C:\Prometheus\fw.ps1


Name                          : {ac85e8fe-8673-4a28-8b2d-07a63ea83340}
DisplayName                   : Prometheus windows_exporter
Description                   :
DisplayGroup                  :
Group                         :
Enabled                       : True
Profile                       : Any
Platform                      : {}
Direction                     : Inbound
Action                        : Allow
EdgeTraversalPolicy           : Block
LooseSourceMapping            : False
LocalOnlyMapping              : False
Owner                         :
PrimaryStatus                 : OK
Status                        : La règle a été analysée à partir de la banque. (65536)
EnforcementStatus             : NotApplicable
PolicyStoreSource             : PersistentStore
PolicyStoreSourceType         : Local
RemoteDynamicKeywordAddresses : {}
```

3. Déclarer sur Debian

Dans le fichier prometheus.yml **(nano /etc/prometheus/prometheus.yml)** on va ajouter le pc avec son nom et son adresse ip avec son port.

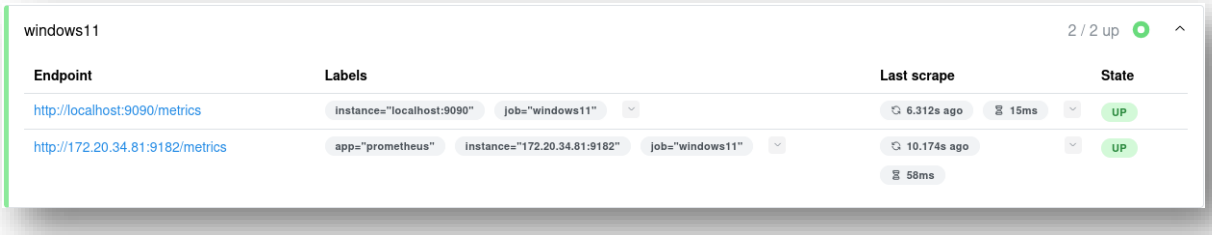
C'est le nom du premier ordinateur qui se trouve dans le secteur Chimie, au rez-de-chaussée dans la première salle.

CH	0	1	1
CHIMIE	Étage	Salle	Numero du pc

```
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"
  - job_name: "windows11"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]
      - targets:
        - "172.20.34.81:9182"
```

Ensuite on fait : http://172.20.34.80:9090/targets

On voit que Windows 11 est UP.



Pour vérifier que ça fonctionne, je vais chercher le nom du pc sur promql : **windows_os_hostname**

Je dois tomber sur **CH011** et c'est bien le cas. La supervision a bien fonctionné.

windows_os_hostname{**app**="prometheus", **fqdn**="CH011", **hostname**="CH011", **instance**="172.20.34.81:9182", **job**="windows11"}

On a réussi **à superviser un pc du bâtiment de chimie**.

VI – Mettre des alertes

1. Configuration du ficher

L'alerting Prometheus est indispensable pour passer d'une observation subie à **une gestion proactive**. Cela va permettre d'agir directement dès que le seuil critique est dépassé.

On pourra prévenir de la panne avant que l'anomalie avant l'arrêt du service.

Dans le répertoire alerts.yml, on va créer nos alertes qui apparaîtront directement sur notre service Prometheus.

Commande : nano /etc/prometheus/alerts.yml

```
root@debian13:~# nano /etc/prometheus/alerts.yml
```

On va créer un groupe au nom de : windows11_monitoring avec 3 règles.

- **Changement de nom**
- **Changement d'adresse ip**
- **Le pc est éteint**

```yaml
groups:
- name: windows11_monitoring
  rules:
  # Changement HOSTNAME Windows 11
  - alert: WindowsHostnameChanged
    expr: changes(wmi_os_computer_system_name[5m]) > 0
    for: 30s
    labels:
      severity: critical
    annotations:
      summary: "HOSTNAME Windows 11 changé : {{ $labels.instance }}"
      description: "Nom PC Windows changé sur {{ $labels.instance }}"
```

```yaml
  #  Changement IP (via target up/down)
  - alert: WindowsIPChanged
    expr: changes(up{job=~"windows.*"}[5m]) > 0
    for: 30s
    labels:
      severity: critical
    annotations:
      summary: "IP Windows 11 changé : {{ $labels.instance }}"
      description: "IP réseau ou connectivité Windows modifiée"
```

```
#  Windows DOWN
- alert: WindowsServerDown
  expr: up{job=~"windows.*"} == 0
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: " CLIENT Windows 11 OFFLINE : {{ $labels.instance }}"
```

On va définir les permissions de fichiers, afin que prometheus puisse fonctionner correctement avec toute sécurité.

Chown = Change Owner, l'outil qui modifie à qui appartient un fichier.

Prometheus = le nom de **l'utilisateur** à qui on donne la propriété du fichier.

: = Séparateur entre l'utilisateur et le groupe

Prometheus =le nom du **groupe** à qui on donne la propriété.

/etc/prometheus/alerts.yml = chemin du fichier cible

```
root@debian13:~# chown prometheus:prometheus /etc/prometheus/alerts.yml
```

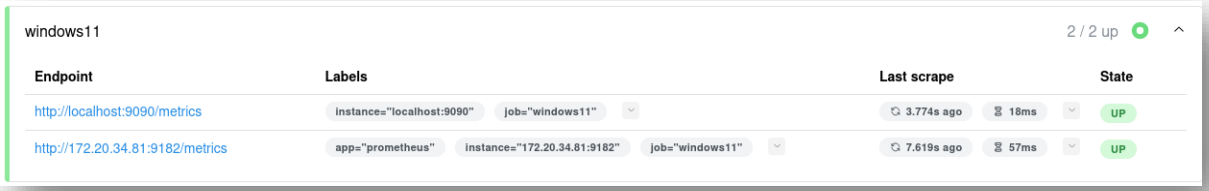
On redémarre le service prometheus.

Commande : systemctl restart prometheus

```
root@debian13:~# systemctl restart prometheus
```

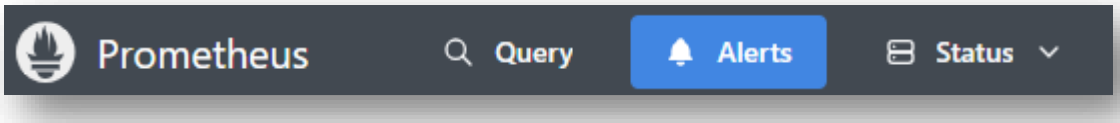
2. Sur le serveur

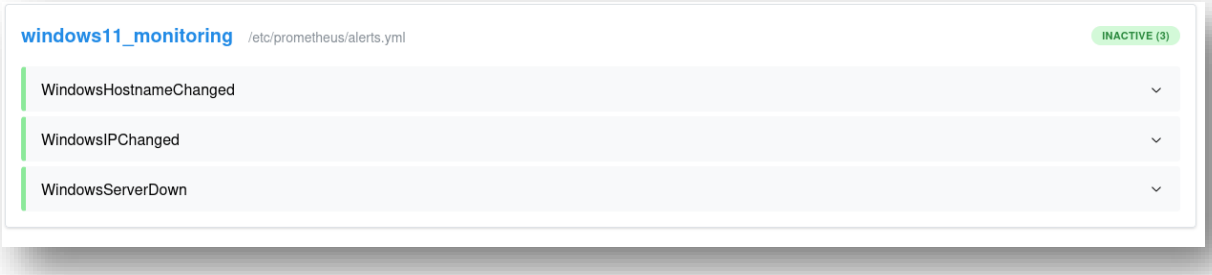
On va se rendre dans notre serveur Prometheus. On peut voir que nos machines sont bien allumées.



On va consulter les **alertes** et voir s'il y'a bien les alertes qu'on a configuré. On peut voir le groupe qu'on a créé *" windows11_monitoring"* et nos 3 règles apparentes.

Elles sont **inactives** pour le moment. Rien ne se passe, tout est ok.

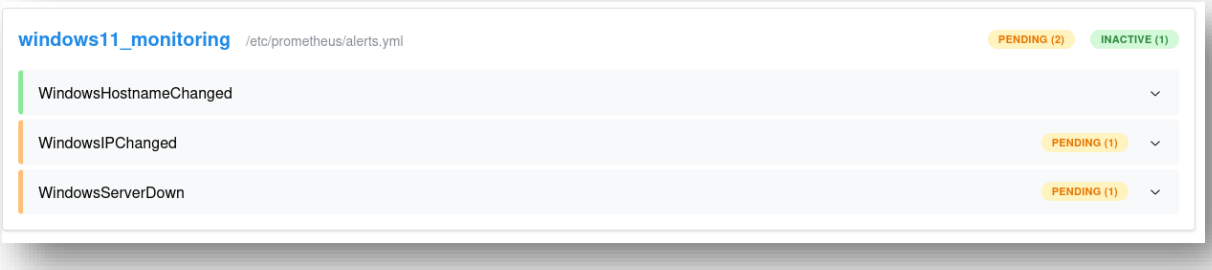




3. Test de règles

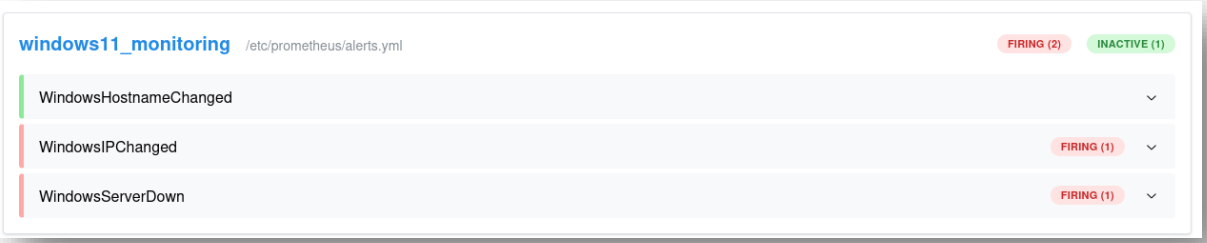
On va faire un test de recette pour voir si nos règles sont bien fonctionnelles et relier à notre ordinateur. On **va changer l'adresse Ip de notre pc** de façon temporaire.

On a changé l'adresse ip de notre ordinateur, on le mettant en DHCP. **Nos alertes passent au jaune.** Il montre qu'il y a des alertes.

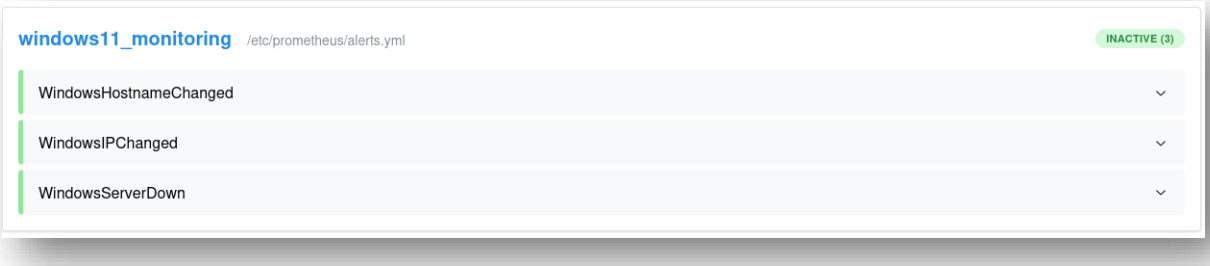


Par le changement d'adresse IP, notre service de supervision ne reconnait plus l'ordinateur donc l'exprime comme un ordinateur éteint. Le pc passe dont au rouge.

Les alertes qui étaient en jaune n'étant pas changé. **Elles passent donc au rouge.** L'ordinateur est reconnu comme éteint et l'adresse ip changée.



Lorsque qu'on remet la bonne adresse ip, toutes **les alertes reviennent en vert.**



VII – Grafana

Grafana est l'outil de **restitution de données.** Son rôle est de se connecter à Prometheus en tant que **Source de données** pour transformer les requêtes complexes en graphiques visuels.

1. Configuration

Tout d'abord, on va installer wget qui est un utilitaire en ligne de commande qui permet de télécharger des fichiers depuis des serveurs web.

Commande : apt-get install –y apt-transport-https wget

```
root@debian13:~# sudo apt-get install -y apt-transport-https wget
```

On va créer un dossier parents avec le chemin standard où sont **stocké les clefs GPG** des dépôts tiers.

Commande : mkdir –p /etc/apt/keyrings/

```
root@debian13:~#  mkdir -p /etc/apt/keyrings/
```

On va **télécharger une clef de confiance de Grafana** et **l'enregistrer** dans le dossier qu'on a créé.

Commande : wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null

```
root@debian13:~# wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
```

On va ajouter le dépôt de Grafana, tout en indiquant à notre service où se trouve le logiciel Grafana.

Commande : echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee /etc/apt/sources.list.d/grafana.list

```
root@debian13:~# echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee -a /etc/apt/sour
ces.list.d/grafana.list
```

Ensuite, on va installer le paquet mis à jour dans la liste de logiciels disponibles puis installer Grafana.

Commande : apt-get update
Commande : apt-get install grafana

```
root@debian13:~# sudo apt-get update
```

```
root@debian13:~# sudo apt-get install grafana
```

Enfin, on lance manuellement et demande à Linux de le lancer à chaque redémarrage.

Commande : systemctl start grafana-server
Commande : systemctl enable grafana-server

```
root@debian13:~#  systemctl enable grafana-server
Synchronizing state of grafana-server.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable grafana-server
Created symlink '/etc/systemd/system/multi-user.target.wants/grafana-server.service' → '/usr/lib/systemd/system/grafana-server.servic
e'.
```

```
root@debian13:~# systemctl start grafana-server
```

Pour voir s'il est bien activé, on va regarder son status.

Commande : systemctl status grafana-server

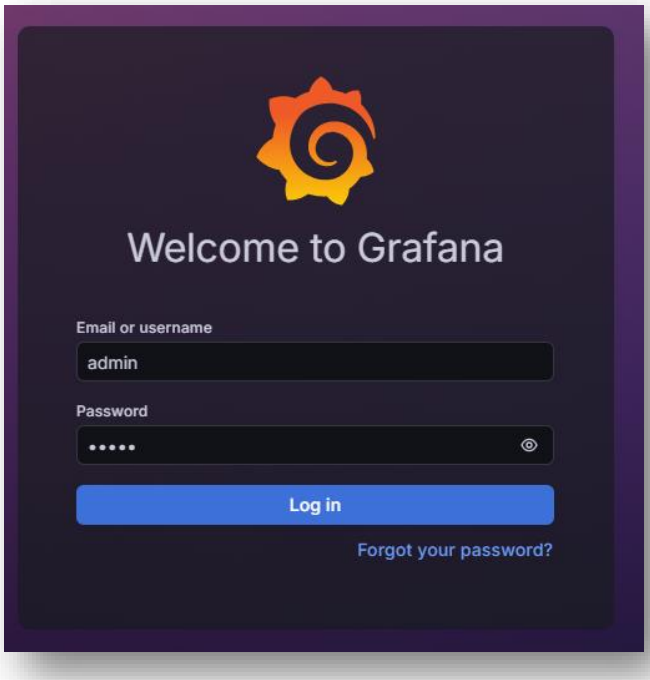
```
root@debian13:~# systemctl status grafana-server
● grafana-server.service - Grafana instance
     Loaded: loaded (/usr/lib/systemd/system/grafana-server.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-01-15 12:45:29 CET; 10s ago
 Invocation: a604bc1c15414f74b8172236e1da98e7
       Docs: http://docs.grafana.org
```

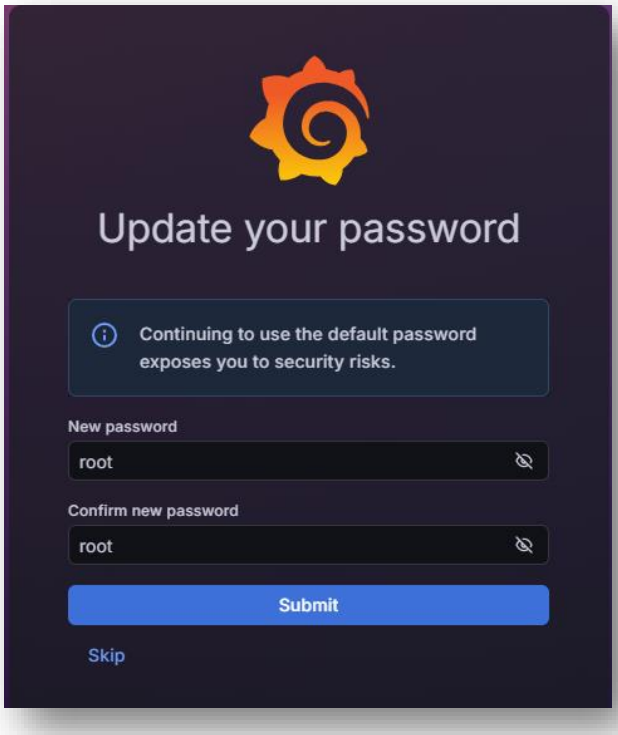
2. Interface du service

Sur le navigateur de notre choix, on met l'adresse ip de notre service (**172.20.34.80**) et le port de Grafana qui est **3000** par défaut afin d'accéder à notre interface graphique.

La première connexion se fait avec le nom et le login : **admin** pour la première fois. Il est recommandé de le changer et de se créer un compte administrateur.

Directement après, on nous demande de changer de mot de passe. On met le mot de passe **root** comme exemple. Mais il faut suivre les recommandations de l'ANSSI donc 14 caractères de majuscules, minuscules, caractères spéciaux et chiffre.



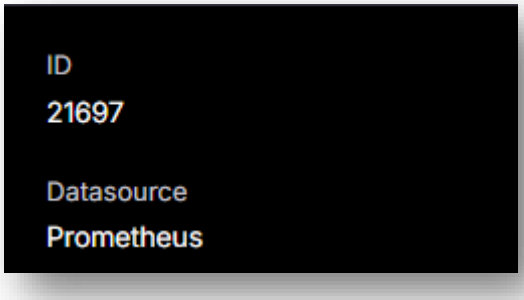


3. Importation du Dashboard

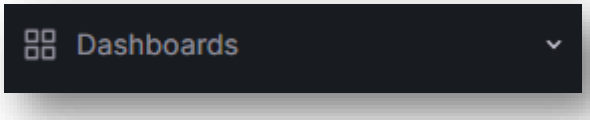
Lien : https://grafana.com/grafana/dashboards/21697-windows-exporter-dashboard/

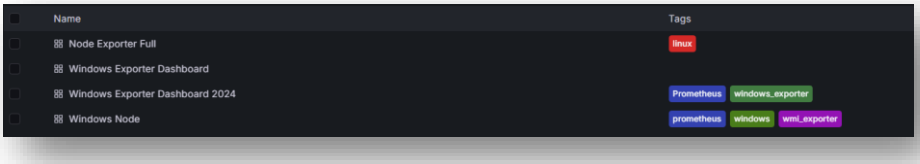
On va choisir notre **restitution de données(Dashboard)** il faut faire attention sur le choix. Il faut choisir, pour le bon système d'exploitation de la machine supervisée (Windows) et non le superviseur (Linux)

On sélectionne l'id de notre Dashboard : **21697** qu'on va installer sur notre grafana.

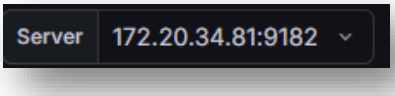


On va sur l'onglet Dashboard, on peut voir tous les exemplaires qu'on a importé ainsi que leur système d'exploitation mis en avant.





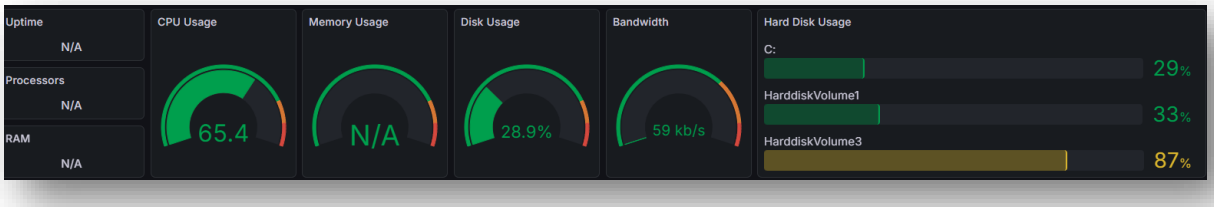
On va suivre la supervision de notre pc. On le met en tant que server, on n'oublie pas l'adresse ip et son port.



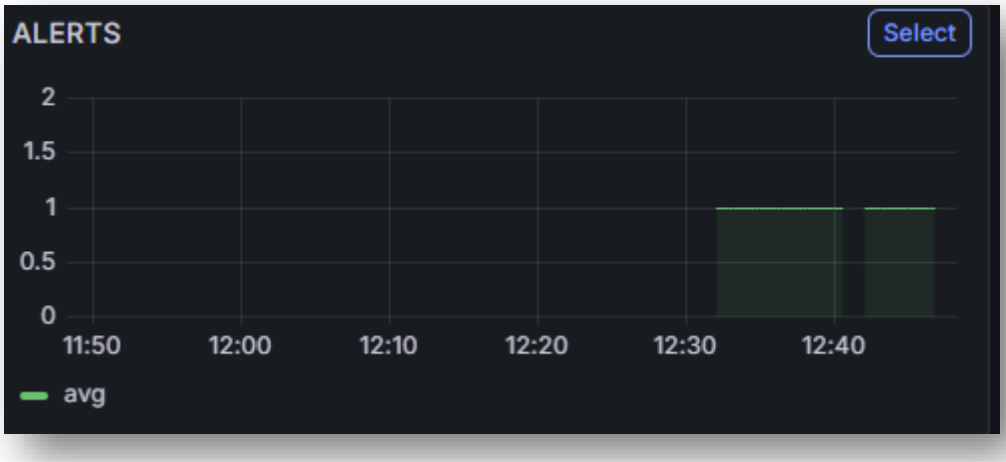
4. Tableau de bord

Grafana transforme des chiffres bruts et compliqués en dessins faciles à lire. C'est un outil de visualisation de données.

<u>Exemple de schéma</u> :



LE POURCENTAGE DE DE LA CHARGE CPU

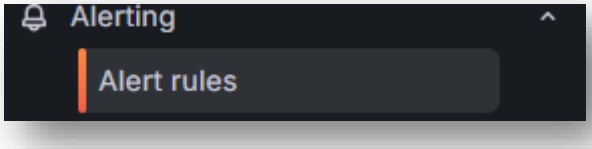
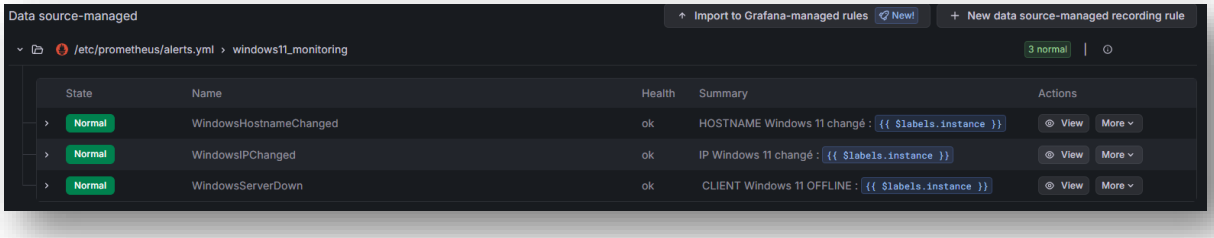


ALERTES QUI SONT LANCEES

TOUTES LES ALERTES AVEC QUI A LANCE L'ALERTE

GARBAGE COLLECTOR PROGRAMME AUTOMATIQUE NETTOIE LA MEMOIRE VIVE

SURVEILLER LA SANTE DE L'APPLICATION

5. Alertes

On peut voir les alertes qu'on a mis sur Prometheus ainsi que leur metrics.